

FCM II Graded Homework 2

Charles Ohanian

February 23, 2025

1 Executive Summary

This report examines and evaluates interpolation techniques, specifically Barycentric 1, Barycentric 2, and Newton interpolation, in terms of accuracy, conditioning, and numerical stability. Various interpolation routines were implemented and tested on different functions and mesh types, including uniform and Chebyshev nodes of the first and second kind. The performance of these methods was analyzed through empirical validation, assessing errors, condition numbers, and relative stability. Results confirm that Chebyshev meshes significantly improve interpolation accuracy and stability, while uniform meshes often suffer from Runge's phenomenon. The study highlights the importance of node selection in achieving numerically stable interpolation.

2 Statement of the Problem

The goal of this report is to analyze different interpolation methods and assess their effectiveness in approximating functions. Specifically, we investigate Barycentric 1, Barycentric 2, and Newton interpolation techniques, comparing their computational efficiency, accuracy, and numerical stability. The study evaluates how different node distributions, including uniform and Chebyshev meshes, impact interpolation error and conditioning. Additionally, we explore the effects of polynomial degree on accuracy and investigate the phenomenon of numerical instability in interpolation. By computing error metrics, condition numbers, and relative errors, we aim to identify the best interpolation strategies for stable and reliable function approximation.

3 Description of the Mathematics

This section outlines the mathematical concepts which will be of use for the construction of and evaluation of interpolation routines. In general, polynomial interpolation seeks a polynomial $p_n(x)$ of degree at most n that exactly matches a set of $n + 1$ data points (x_i, y_i) , where $y_i = f(x_i)$ for some underlying function f . This report will use two main interpolation techniques: Lagrange interpolation and Newton interpolation.

3.1 Lagrange Interpolation

When utilizing the Lagrange interpolation method, the interpolating polynomial takes the following form:

$$p_n(x) = \sum_{i=0}^n f_i \ell_i(x), \quad (1)$$

where the Lagrange basis functions $\ell_i(x)$ are defined as

$$\ell_i(x) = \frac{\prod_{k=0, k \neq i}^n (x - x_k)}{\prod_{k=0, k \neq i}^n (x_i - x_k)}. \quad (2)$$

3.2 Newton Interpolation

Alternatively, the Newton interpolation formula constructs $p_n(x)$ in a recursive manner:

$$\begin{aligned} p_n(x) = & f[x_0] + f[x_0, x_1](x - x_0) + f[x_0, x_1, x_2](x - x_0)(x - x_1) + \dots \\ & + f[x_0, x_1, \dots, x_n](x - x_0)(x - x_1) \cdots (x - x_{n-1}). \end{aligned} \quad (3)$$

Here, the divided differences are recursively defined as follows:

$$f[x_j, x_{j+1}, \dots, x_k] = \frac{f[x_{j+1}, \dots, x_k] - f[x_j, \dots, x_{k-1}]}{x_k - x_j}, \quad (4)$$

with the initial values given by $f[x_i] = f_i$.

3.3 Barycentric Interpolation

Due to several key issues, such as high computational cost, numerical instability, and the difficulty of adding or modifying an existing mesh point, the Lagrange for is rarely used in practice. Instead, the Barycentric interpolation method is utilized. The first form of the Barycentric interpolation is given by:

$$p_n(x) = \omega_{n+1}(x) \sum_{i=0}^n y_i \frac{\gamma_i}{(x - x_i)}, \quad (5)$$

where

$$\gamma_i = \frac{1}{\omega'_{n+1}(x_i)} \quad \text{and} \quad \omega_{n+1}(x) = \prod_{i=0}^n (x - x_i). \quad (6)$$

A more commonly used second form of Barycentric interpolation, where the weights depend on the type of mesh $\{x_i\}$, is defined as follows:

$$p_n(x) = \frac{\sum_{i=0}^n y_i \frac{\gamma_i}{(x - x_i)}}{\sum_{i=0}^n \frac{\gamma_i}{(x - x_i)}}, \quad (7)$$

The γ_i 's can be replaced by β_i in the above formula via cancellation of a common factor. For a mesh of uniformly spaced points on the interval $[a, b]$, the weights are recursively defined as:

$$\beta_{i+1} = -\beta_i \frac{n-i}{i+1}, \quad \text{with } \beta_0 = 1. \quad (8)$$

If the interpolation nodes are Chebyshev points of the first kind, given by

$$x_j = \cos \left(\frac{(2j+1)\pi}{2n+2} \right), \quad 0 \leq j \leq n, \quad (9)$$

then the weights are:

$$\beta_i = (-1)^j \sin \left(\frac{(2j+1)\pi}{2n+2} \right) \quad 0 \leq j \leq n. \quad (10)$$

For Chebyshev points of the second kind,

$$x_j = \cos \left(\frac{j\pi}{n} \right), \quad 0 \leq j \leq n, \quad (11)$$

the weights are given by

$$\beta_i = (-1)^j \delta_j, \quad \delta_j = \begin{cases} 1/2, & j = 0 \text{ or } j = n \\ 1, & \text{otherwise} \end{cases}. \quad (12)$$

3.4 Horner's Rule for Newton Form

Horner's Rule is an efficient recursive approach for computing the Newton form of an interpolating polynomial. As an example, let $n = 4$. Then we have:

$$p_4(x) = \alpha_0 + \alpha_1(x-x_0) + \alpha_2(x-x_0)(x-x_1) + \alpha_3(x-x_0)(x-x_1)(x-x_2) + \alpha_4(x-x_0)(x-x_1)(x-x_2)(x-x_3), \quad (13)$$

where $\alpha_j = f[x_0, x_1, \dots, x_j]$. This polynomial can be restructured to minimize computations as follows:

$$p_4(x) = (((\alpha_4(x-x_3) + \alpha_3)(x-x_2) + \alpha_2)(x-x_1) + \alpha_1)(x-x_0) + \alpha_0. \quad (14)$$

3.5 Condition Numbers

Condition numbers measure how sensitive the interpolating polynomial is to small changes in the input data. A high condition number indicates that even small perturbations in the data can cause large changes in the resulting interpolant, leading to numerical instability. Two forms of condition number will be computed in this report, $\kappa(x, n, y)$ and $\kappa(x, n, 1)$. For Barycentric 1, we have:

$$\kappa(x, n, y) = \frac{\sum_{i=0}^n |\ell_i(x) y_i|}{|p_n(x)|}. \quad (15)$$

For Barycentric 2, the condition number is given in Higham (2004) as follows:

$$\frac{\sum_{j=0}^n \left| \frac{\beta_j y_j}{x - x_j} \right|}{\left| \sum_{j=0}^n \frac{\beta_j y_j}{x - x_j} \right|}. \quad (16)$$

To obtain $\kappa(x, n, 1)$ for either form, simply substitute 1 in for y_i , i.e. for Barycentric 1:

$$\kappa(x, n, 1) = \sum_{i=0}^n |\ell_i(x)|. \quad (17)$$

4 Description of the Algorithm and Implementation

All routines and experiments were developed and run in Google Colab, a cloud-based platform provided by Google that allows users to write and execute Python code in a Jupyter Notebook environment.

This section will accomplish the first part of Task 1 - the design of the codes and their complexity with respect to time and space will be discussed. The empirical validation of the routines will be left to the beginning of the results section.

4.1 Interpolation Routines

Routines were developed for Barycentric form 1 interpolation, Barycentric form 2 interpolation, and Newton interpolation using an adaptation of Horner's rule. All of these routines are built with the ability to run in either single or double precision. This will become of use for testing purposes later.

The Barycentric 1 interpolation routine consists of two primary functions: `bary_1_gammas`, which computes the Barycentric weight coefficients (γ_i) and function values at given mesh points, and `bary_1_interpolation`, which evaluates the interpolating polynomial at given x points using the precomputed weights. The `bary_1_gammas` function initializes an array to store the Barycentric weights and iterates through each interpolation node x_i , computing the product: $\omega'_{n+1} = \prod_{j \neq i}^n (x_i - x_j)$, and setting $\gamma_i = \frac{1}{\omega'_i}$. Additionally, it evaluates the function at the given mesh points. The time complexity for this function is $O(n^2)$, since computing each ω'_i requires iterating over all $n + 1$ points. The space complexity is $O(n)$, as it only requires storage for the weights and function values. The `bary_1_interpolation` function takes these precomputed values and computes interpolated values for given x points. It first converts input arrays to the required precision and then iterates over each x point x_j . If an x point is numerically close to a mesh point, the function value at that mesh point is directly used. Otherwise, it computes the weight factor: $\omega_{n+1}(x) = \prod_{i=0}^n (x - x_i)$, and the weighted sum (Barycentric 1 formula):

$$p_n(x) = \omega_{n+1}(x) \sum_{i=0}^n \frac{y_i \gamma_i}{x - x_i}.$$

It also computes conditioning factors to later analyze stability. The time complexity to evaluate the interpolant at a given x point is $O(n)$. The space complexity is also $O(n)$, as it requires storage for precomputed values, interpolant results, and conditioning terms.

The Barycentric 2 interpolation routine consists of two primary functions: `bary_2_betas`, which computes the Barycentric weight coefficients (β_i) and function values at given mesh points, and `bary_2_interpolation`, which evaluates the interpolating polynomial at given x points using the precomputed weights. The `bary_2_betas` function initializes an array to store the Barycentric weights and iterates through each interpolation node x_i to compute the weight coefficients. Depending on the type of mesh, different strategies are used. For a uniform mesh, the weights are recursively computed as: $\beta_{i+1} = -\beta_i \cdot \frac{n-i}{i+1}$, starting with $\beta_0 = 1$. For Chebyshev nodes of the first kind, the weights are computed using: $\beta_i = (-1)^i \sin\left(\frac{(2i+1)\pi}{2n+2}\right)$. For Chebyshev nodes of the second kind, the weights are $\beta_i = (-1)^i$ for interior points, with the endpoints scaled by $\frac{1}{2}$. The function then evaluates the given function at the mesh points. The time complexity for computing the weights is $O(n)$ for Uniform and Chebyshev 1 meshes and $O(1)$ for the Chebyshev 2 mesh. The space complexity is also $O(n)$, as only the weights and function values are stored. The `bary_2_interpolation` function takes the precomputed values and computes interpolated values for given x points. It first converts input arrays to the required precision and then iterates over each x point x_j . If an x point is numerically close to a mesh point, the function value at that mesh point is directly used. Otherwise, it computes the numerator and denominator in the Barycentric 2 formula:

$$p_n(x) = \frac{\sum_{i=0}^n \frac{y_i \beta_i}{x - x_i}}{\sum_{i=0}^n \frac{\beta_i}{x - x_i}}.$$

It also computes conditioning factors to later analyze stability. The time complexity to evaluate the interpolant at a given x point is $O(n)$. The space complexity is also $O(n)$, as it requires storage for precomputed values, interpolant results, and conditioning terms. The Barycentric 2 method is more efficient than Barycentric 1, reducing weight computation from $O(n^2)$ to $O(n)$. This makes it preferable for large-scale interpolation problems.

The Newton interpolation routine consists of two primary functions: `divided_differences`, which computes the divided difference coefficients, and `horner_interpolation`, which evaluates the interpolating polynomial using Horner's rule. The `divided_differences` function first computes function values at the mesh points and stores them in an array. It then initializes a divided difference coefficients array with the function values. The algorithm progresses through increasing orders of divided differences, from first-order to $(n - 1)$ th order. Each iteration corresponds to computing a new level in the divided difference table. An inner loop moves backwards through the coefficients array to update higher-order divided differences using the difference of two previously computed divided differences from the earlier iteration. This process requires $O(n^2)$ operations since each divided difference computation depends on previous values. The space complexity is $O(n)$ since the routine only maintains a 1D array of function values and a 1D array of divided difference coefficients. The `horner_interpolation` function evaluates the Newton interpolation polynomial using Horner's rule. It first retrieves the array of coefficients from the previous routine and then iterates through each of the given x points to evaluate the interpolant. As Horner's rule is utilized, the interpolant is originally

set to the last coefficient and then updated as the algorithm moves backwards through the coefficients. This method ensures numerical stability and requires only $O(n)$ operations per evaluation. The space complexity remains $O(n)$ as only a few temporary variables are used during computation.

Table 1 provides a summary of time and space complexity for the aforementioned interpolation algorithms.

Method	Construction	Evaluation	Space Complexity
Barycentric 1	$O(n^2)$	$O(n)$	$O(n)$
Barycentric 2	$O(n)$ or $O(1)$	$O(n)$	$O(n)$
Newton	$O(n^2)$	$O(n)$	$O(n)$

Table 1: Time and Space Complexity of Different Interpolation Methods

4.2 Supplementary Algorithms

The first supplementary algorithm is a routine that generates the three types of meshes, titled `create_mesh`. This function generates a set of n mesh points within the specified interval $[a, b]$ based on the specified flag. If flag is 1, it creates a uniform mesh using `np.linspace`, distributing points evenly across the interval. If flag is 2, it generates Chebyshev nodes of the first kind, which are computed using the cosine function to cluster points near the endpoints, improving interpolation accuracy. If flag is 3, it constructs Chebyshev nodes of the second kind, which include the interval endpoints and also distribute points in a way that minimizes interpolation errors. The function returns the computed mesh as a numpy array.

The second supplementary algorithm is a routine that alters the ordering of an inputted mesh, titled `mesh_order`. This function reorders a given set of distinct mesh points based on the specified flag. If flag is 1, it sorts the points in increasing order using a bubble sort algorithm. If flag is 2, it sorts them in decreasing order, also using bubble sort. If flag is 3, it arranges the points in a Leja ordering. This ordering begins by selecting the point with the largest absolute value and iteratively choosing subsequent points to maximize the product of absolute differences between selected points and all previously chosen ones. The function returns the reordered mesh.

Algorithms to compute error statistics were also included. The `compute_max_norm` function calculates the infinity norm between two input arrays x and y by computing the maximum absolute difference between their corresponding elements. The `relative_error` function computes the element-wise relative error between two arrays, p_n and $\hat{p}_n$, by dividing the absolute difference by the absolute value of p_n . Here, p_n is intended to be the "exact" value of the interpolant computed in double precision and $\hat{p}_n$ is intended to be the computed value of the interpolant computed in single precision.

5 Description of the Experimental Design and Results

5.1 Task 1

For a description of the design of your codes and a discussion of the complexity with respect to time and space, see Section 4. Here, we will provide the results of basic tests to empirically validate the interpolation routines.

We interpolate $f_1(x) = (x-2)^9$ with all three interpolation routines (Barycentric 1, Barycentric 2, and Newton). A range of meshes was used to demonstrate the versatility of the algorithms. In all three cases, we interpolate with a 15th degree polynomial, meaning there are 16 interpolation nodes. All three tests interpolate 100 equally spaced points on the interval. The intervals are varied for the three tests simply to increase the chance of seeing error in the algorithms. Figure 1 shows the results of these tests. The exact $f_1(x)$ is plotted, along with the interpolated polynomial and the 16 interpolation points. It can be seen that in all three tests, the interpolated polynomial matches very well with $f_1(x)$. Table 1 displays the maximum pointwise error for each of the three tests. It is evident that the interpolation is more accurate in the Barycentric 2 and Newton tests, however, the slight increase in error in the Barycentric 1 case is due to the larger $f(x)$ values because of the wider interval and high-degree function. Given the degree of interpolating polynomial, the error is small enough in all cases to conclude that the interpolation algorithms work as desired.

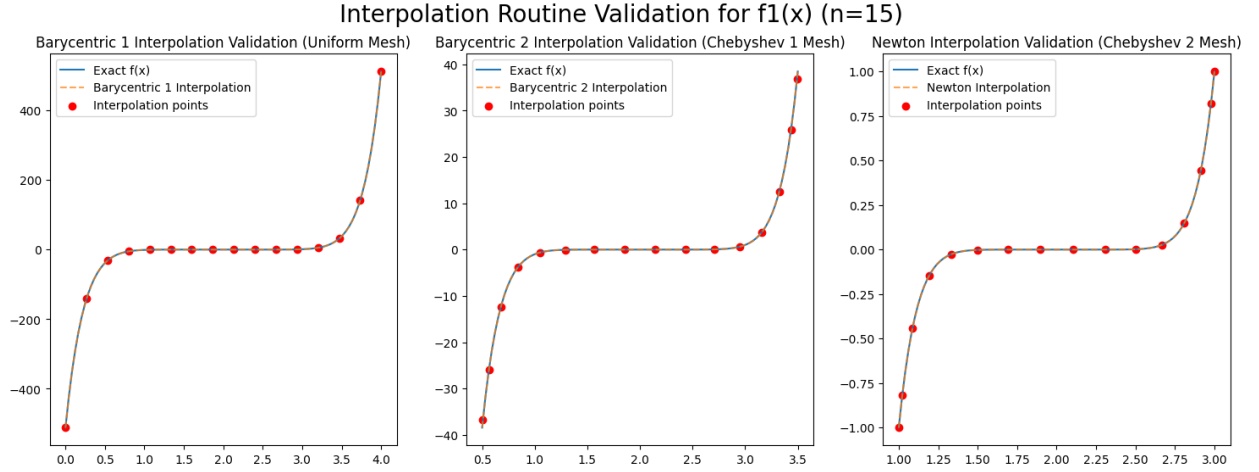


Figure 1. $p_{15}(x)$ interpolating $f_1(x) = (x-2)^9$. Left plot shows barycentric 1 interpolation with a uniform mesh. Middle plot shows barycentric 2 interpolation with a Chebyshev 1 mesh. Right plot shows Newton interpolation with a Chebyshev 2 mesh. Note the differing intervals.

Method / Mesh	Maximum Pointwise Error ($\ f(x) - p_n(x)\ _\infty$)
Barycentric 1 (Uniform)	0.0001488357446532973
Barycentric 2 (Chebyshev 1)	$1.654063193257116 \times 10^{-5}$
Newton (Chebyshev 2)	$4.623930786712416 \times 10^{-5}$

Table 2: Maximum Pointwise Error for Each Interpolation Routine

5.2 Task 2

In this section, interpolation accuracy, conditioning, and stability is discussed for $f_1(x) = (x - 2)^9$.

5.2.1 Interpolation Accuracy

Figure 2 shows attempts to interpolate $f_1(x)$ with $p_4(x)$ using Barycentric 1, Barycentric 2, and Newton interpolation methods. Newton is tested with all three mesh orderings and all tests incorporate all three mesh types: Uniform points, Chebyshev points of the first kind, and Chebyshev points of the second kind. The interpolant is evaluated at 100 equally spaced points on the common interval $[0.5, 3.5]$. It can be easily observed that $p_4(x)$ does not accurately interpolate $f_1(x)$. Table 3 shows the maximum pointwise error for each method and mesh. Interestingly, no matter what method is used, the maximum error is the same to at least 4 digits past the decimal point for each type of mesh. Evidently, the fundamental limitation in this scenario is the degree of the interpolating polynomial rather than the method used. We are attempting to interpolate a 9th degree polynomial with a 4th degree polynomial, which will never work as $4 < 9$. This means that the dominant source of error is the degree mismatch, not the choice of interpolation method or mesh type.

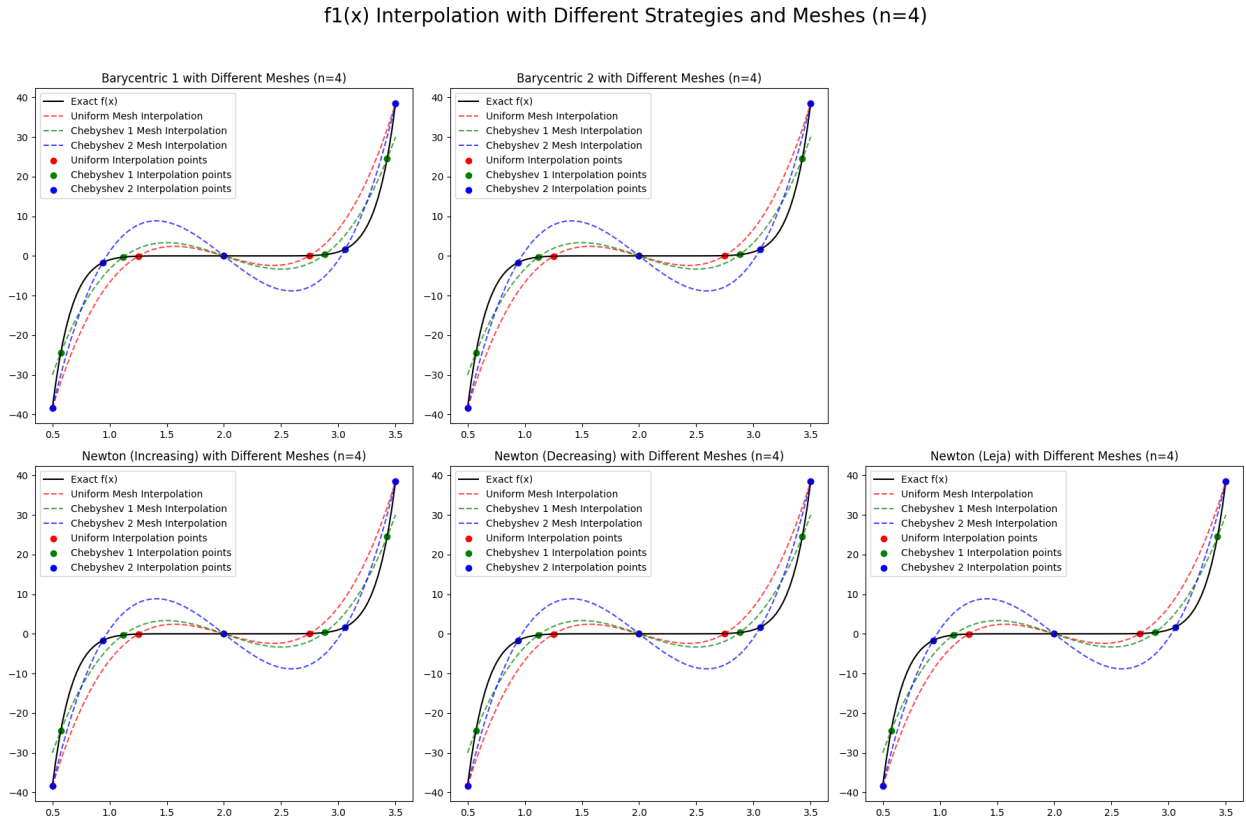


Figure 2. $p_4(x)$ interpolating $f_1(x) = (x - 2)^9$. Plots show Barycentric 1, Barycentric 2, Newton interpolation using all 3 mesh types. Newton interpolation is tested for all 3 mesh orders.

Method / Mesh	Maximum Pointwise Error ($\ f(x) - p_n(x)\ _\infty$)
Barycentric 1 (Uniform)	11.738057018705407
Barycentric 1 (Chebyshev 1)	8.409479141235352
Barycentric 1 (Chebyshev 2)	8.85341983634643
Barycentric 2 (Uniform)	11.738060833402688
Barycentric 2 (Chebyshev 1)	8.409482955932617
Barycentric 2 (Chebyshev 2)	8.85341983634643
Newton Increasing (Uniform)	11.738058926054055
Newton Increasing (Chebyshev 1)	8.40948486328125
Newton Increasing (Chebyshev 2)	8.853418882672113
Newton Decreasing (Uniform)	11.738057018705407
Newton Decreasing (Chebyshev 1)	8.409481048583984
Newton Decreasing (Chebyshev 2)	8.853426512066644
Newton Leja (Uniform)	11.738060833402688
Newton Leja (Chebyshev 1)	8.409481048583984
Newton Leja (Chebyshev 2)	8.853422697369378

Table 3: Maximum Pointwise Error for Each Interpolation Routine (n=4)

Now, we try the same experiment, but use 10 mesh points rather than 4. That is, we attempt to interpolate $f_1(x)$ with $p_{10}(x)$. A quick look at Figure 3 demonstrates that the interpolation is significantly more accurate with this higher degree interpolating polynomial. The low maximum pointwise errors displayed in Table 4 demonstrate the higher accuracy seen in these tests. This is to be expected as the degree of the interpolating polynomial is greater than the degree of $f(x)$. It is known that $E_n(x) = f(x) - p_n(x) = 0$ if $f(x) \in \mathbb{P}_n$, which is the case here.

f₁(x) Interpolation with Different Strategies and Meshes (n=10)

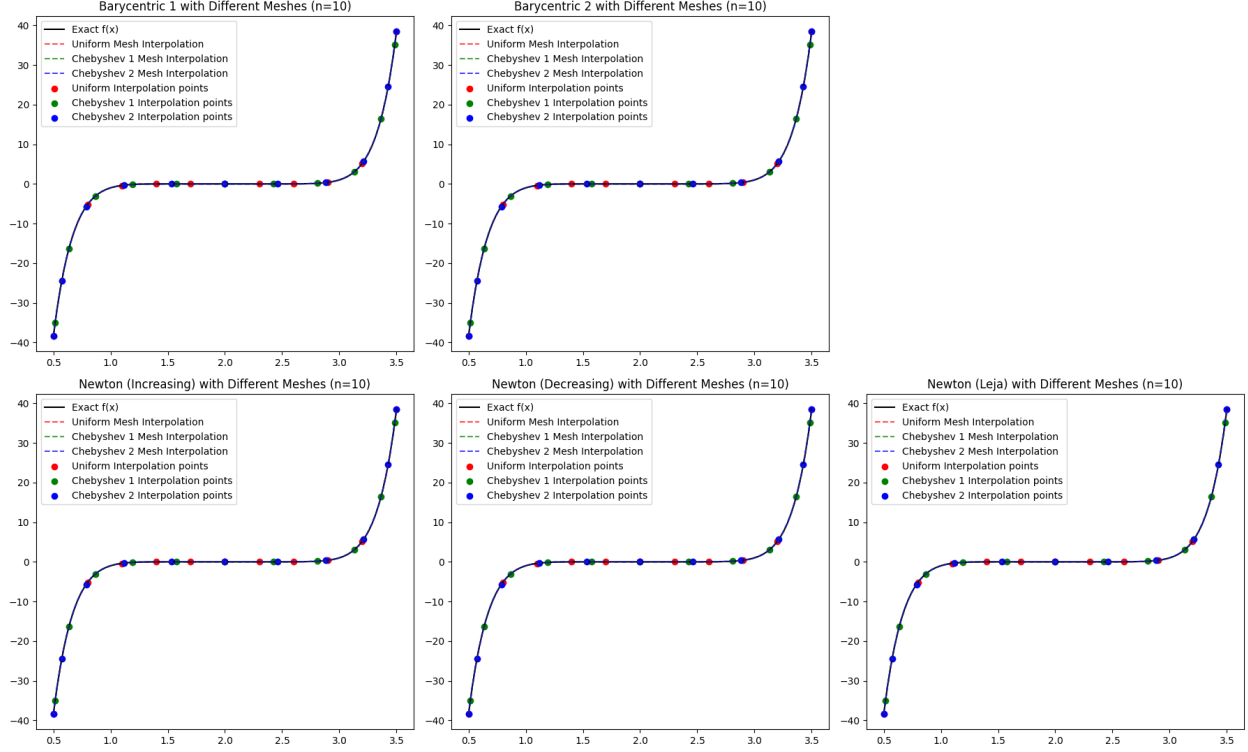


Figure 3. $p_{10}(x)$ interpolating $f_1(x) = (x - 2)^9$. Plots show Barycentric 1, Barycentric 2, Newton interpolation using all 3 mesh types. Newton interpolation is tested for all 3 mesh orders.

Method / Mesh	Maximum Pointwise Error ($\ f(x) - p_n(x)\ _\infty$)
Barycentric 1 (Uniform)	1.312757×10^{-5}
Barycentric 1 (Chebyshev 1)	2.217845×10^{-5}
Barycentric 1 (Chebyshev 2)	2.266431×10^{-5}
Barycentric 2 (Uniform)	3.029371×10^{-5}
Barycentric 2 (Chebyshev 1)	1.525879×10^{-5}
Barycentric 2 (Chebyshev 2)	2.075697×10^{-5}
Newton Increasing (Uniform)	3.503266×10^{-4}
Newton Increasing (Chebyshev 1)	6.910857×10^{-4}
Newton Increasing (Chebyshev 2)	6.384966×10^{-4}
Newton Decreasing (Uniform)	4.899647×10^{-4}
Newton Decreasing (Chebyshev 1)	6.910857×10^{-4}
Newton Decreasing (Chebyshev 2)	6.384966×10^{-4}
Newton Leja (Uniform)	1.694227×10^{-5}
Newton Leja (Chebyshev 1)	1.503492×10^{-5}
Newton Leja (Chebyshev 2)	1.312757×10^{-5}

Table 4: Maximum Pointwise Error for Each Interpolation Routine (n=10)

5.2.2 Conditioning

Conditioning was tested for Barycentric 1 and Barycentric 2 interpolation for all three mesh types. Conditioning was evaluated using $\kappa(x, n, y)$ and $\kappa(x, n, 1)$, as defined in Section 3. For both barycentric methods and for each mesh type, $\kappa(x, n, y)$ is plotted against the x values to be interpolated, as this behavior can be quite interesting. The mean $\kappa(x, n, y)$ over the given interval is noted as a summary statistic. For Barycentric 1, the Lebesgue constant, or the maximum value of $\kappa(x, n, 1)$, is computed as well. The Lebesgue constant is computed and shown for Barycentric 1 because this form directly evaluates the interpolant using Lagrange basis functions, making it highly sensitive to the choice of interpolation nodes and prone to numerical instability for large n . In contrast, Barycentric 2 precomputes the weight coefficients in a way that inherently improves numerical conditioning, making it significantly less affected by the Lebesgue constant. Since Barycentric 2 avoids explicit Lagrange basis evaluations, the impact of the Lebesgue constant is minimized, and thus it is not necessary to compute or analyze it for this form. Figure 4 shows $\kappa(x, n, y)$ for both barycentric methods for different n values, using the uniform mesh. It is clear that $\kappa(x, n, y) \rightarrow \infty$ as $x \rightarrow 2$ in Barycentric 1. This makes sense when looking at the formula for $\kappa(x, n, y)$. Evaluating this number when close to $x = 2$ (the root of the function) will cause division by a number very close to 0. This is amplified as n increases because the interpolation accuracy gets better. However, we do see smaller condition numbers for Barycentric 2, which demonstrates that the function $f_1(x)$ is more numerically stable in Barycentric 2 interpolation when using a uniform mesh. The rapidly increasing Lebesgue constant as n increases, as seen in Table 5 shows that the uniform mesh is incredibly unstable, as expected.

$\kappa(x, n, y)$ using Barycentric 1 and Barycentric 2 with Uniform Mesh for $f_1(x)$

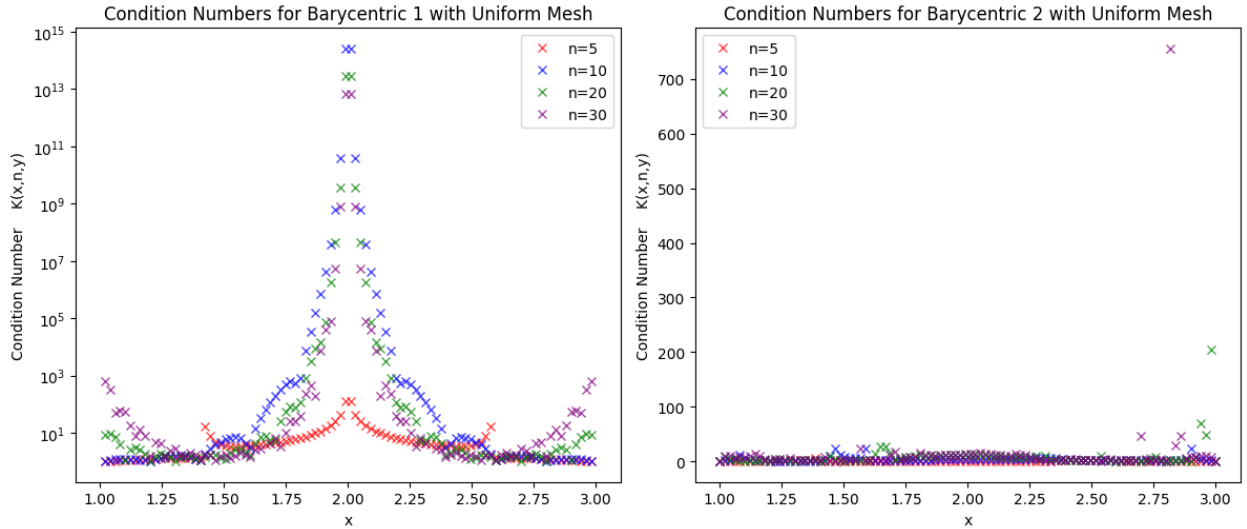


Figure 4. Condition numbers $\kappa(x, n, y)$ for Barycentric 1 and Barycentric 2 interpolation of $f_1(x)$ for different interpolating degrees using the uniform mesh. A log scale is used for the Barycentric 1 plot (left).

Method	$n = 5$	$n = 10$	$n = 20$	$n = 30$
Barycentric 1 Mean $K(x, n, y)$	7.7481	5.15×10^{12}	5.49×10^{11}	1.39×10^{11}
Barycentric 1 Lebesgue Constant	3.1039	29.8970	10714.4240	6276939.0485
Barycentric 2 Mean $K(x, n, y)$	1.1504	3.7082	8.6563	14.8164

Table 5: Comparison of Mean $\kappa(x, n, y)$ and Lebesgue Constant for Barycentric Methods using the Uniform Mesh

Figure 5 shows $\kappa(x, n, y)$ for both barycentric methods for different n values, using the Chebyshev 1 mesh. It is again clear that $\kappa(x, n, y) \rightarrow \infty$ as $x \rightarrow 2$ in Barycentric 1. This is to be expected for the same reasons as discussed above. Again, smaller condition numbers are seen for Barycentric 2, which demonstrates that when using a Chebyshev 1 mesh, the function $f_1(x)$ is more numerically stable in Barycentric 2 interpolation. Unlike with the uniform mesh, the Lebesgue constant remains very low with the Chebyshev 1 mesh, as seen in Table 6. This highlights the great benefits of this mesh (and the dangers of using the uniform mesh) in terms of numerical stability.

$\kappa(x, n, y)$ using Barycentric 1 and Barycentric 2 with Chebyshev 1 Mesh for $f_1(x)$

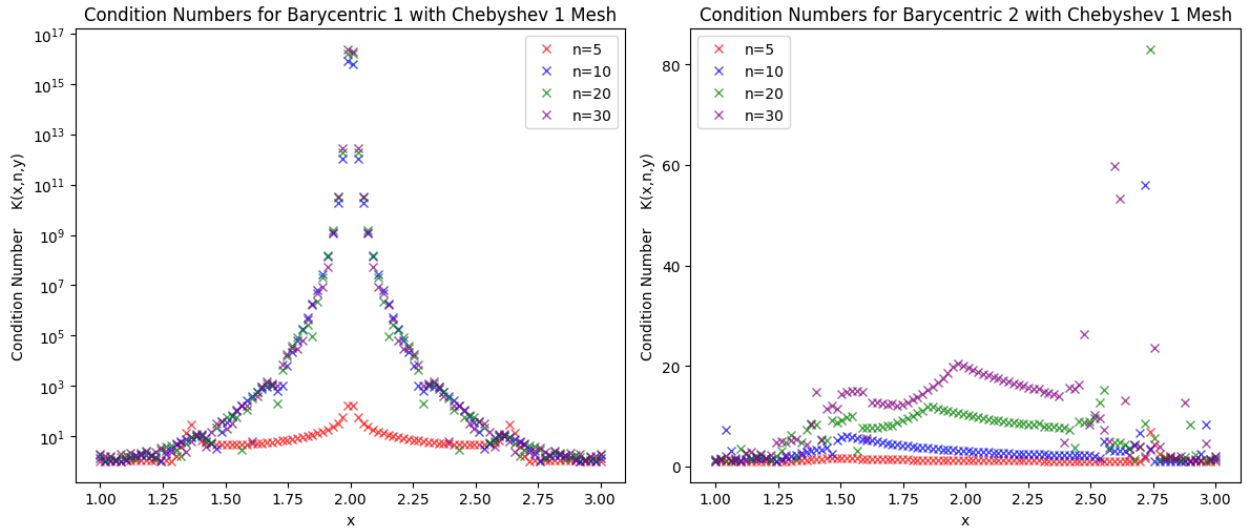


Figure 5. Condition numbers $\kappa(x, n, y)$ for Barycentric 1 and Barycentric 2 interpolation of $f_1(x)$ for different interpolating degrees using the Chebyshev 1 mesh. A log scale is used for the Barycentric 1 plot (left).

Method	$n = 5$	$n = 10$	$n = 20$	$n = 30$
Barycentric 1 Mean $K(x, n, y)$	11.041	1.4440×10^{14}	3.2908×10^{14}	4.2665×10^{14}
Barycentric 1 Lebesgue Constant	2.1044	2.4894	2.9008	3.1487
Barycentric 2 Mean $K(x, n, y)$	1.3202	3.5123	7.3217	11.1954

Table 6: Comparison of Mean $\kappa(x, n, y)$ and Lebesgue Constant for Barycentric methods using the Chebyshev 1 mesh

Figure 6 shows $\kappa(x, n, y)$ for both barycentric methods for different n values, using the Chebyshev 2 mesh. There are clear similarities to the Chebyshev 1 plots in Figure 5. The

Barycentric 1 condition numbers are largely the same and the Barycentric 2 plot looks similar albeit with smaller condition numbers. The high $\kappa(x, n, y)$ for both methods, but especially in Barycentric 1 highlight the ill conditioned nature of $f_1(x)$, especially around the root $x = 2$. The low Lebesgue constants seen in Table 7 demonstrate that the interpolation process itself is quite well-conditioned and numerically stable. When comparing the meshes, it is clear, based on the exponentially increasing Lebesgue constants, that the uniform mesh is to be avoided. Chebyshev 2 looks slightly better than Chebyshev 1 in this case, but the difference is not vast.

k(x,n,y) using Barycentric 1 and Barycentric 2 with Chebyshev 2 Mesh for f1(x)

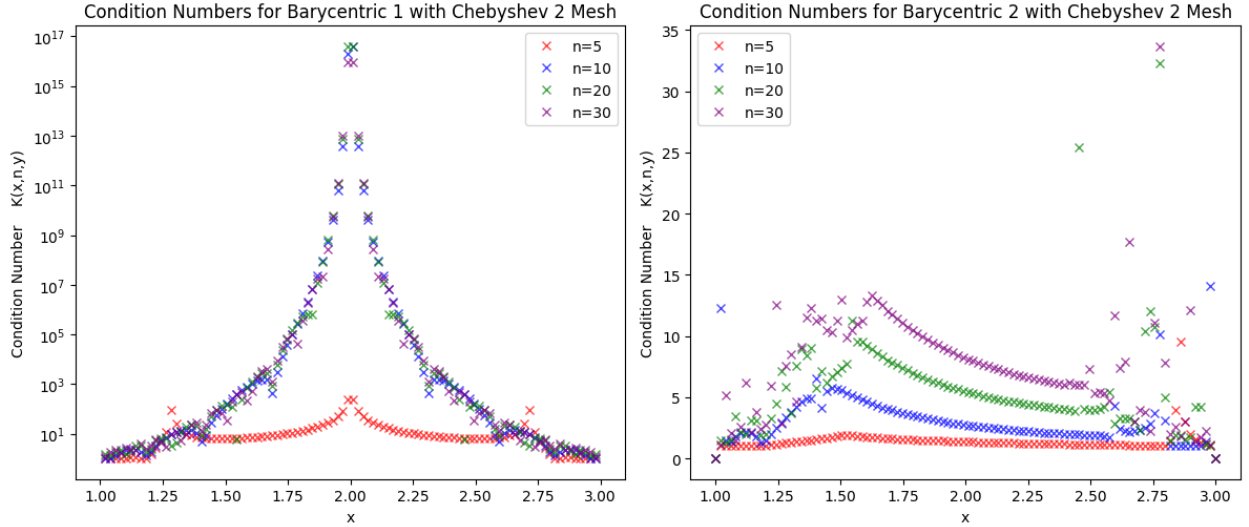


Figure 6. Condition numbers $\kappa(x, n, y)$ for Barycentric 1 and Barycentric 2 interpolation of $f_1(x)$ for different interpolating degrees using the Chebyshev 2 mesh. A log scale is used for the Barycentric 1 plot (left).

Method	$n = 5$	$n = 10$	$n = 20$	$n = 30$
Barycentric 1 Mean $K(x, n, y)$	17.4646	5.9124×10^{14}	7.8837×10^{14}	1.7536×10^{14}
Barycentric 1 Lebesgue Constant	1.9877	2.4196	2.8516	3.1239
Barycentric 2 Mean $K(x, n, y)$	1.4245	3.0939	5.5908	7.5498

Table 7: Comparison of Mean $K(x, n, y)$ and Lebesgue Constant for Barycentric 1 and Barycentric 2 using the Chebyshev 2 mesh

5.2.3 Relative Error and Stability

Relative error is computed as in Higham (2004):

$$e_{\text{rel}} = \frac{|p_n(x) - \hat{p}_n(x)|}{|p_n(x)|},$$

where $\hat{p}_n(x)$ represents the computed interpolating polynomial (single precision) and $p_n(x)$ represents the "exact" interpolating polynomial, computed in double precision. By assessing

relative error in this way, we can identify which interpolation methods and node distributions are more susceptible to numerical instability. Figure 7 shows relative error results for $f_1(x) = (x - 2)^9$. With all interpolation strategies, there is a clear jump in relative error around the root $x = 2$. This makes sense and is to be expected. To compute relative error, the absolute difference is divided by the interpolant evaluated in double precision. When close to the root, this means dividing by a number very close to 0, which causes the relative error to blow up. This again highlights the numerical instability of these routines around the roots of a polynomial. When finding relative error for a lower degree interpolating polynomial, say $n = 6$, it is evident that the relative error is much lower in every test, never rising above 10^{-2} . There do not seem to be very noticeable differences between the mesh types for this function. When testing for a lower degree, the uniform mesh performs slightly worse in all cases than the Chebyshev meshes, but when examining for a large n , the instability caused by the proximity to the root dominates.

Relative Error with Different Strategies and Meshes (n=29)

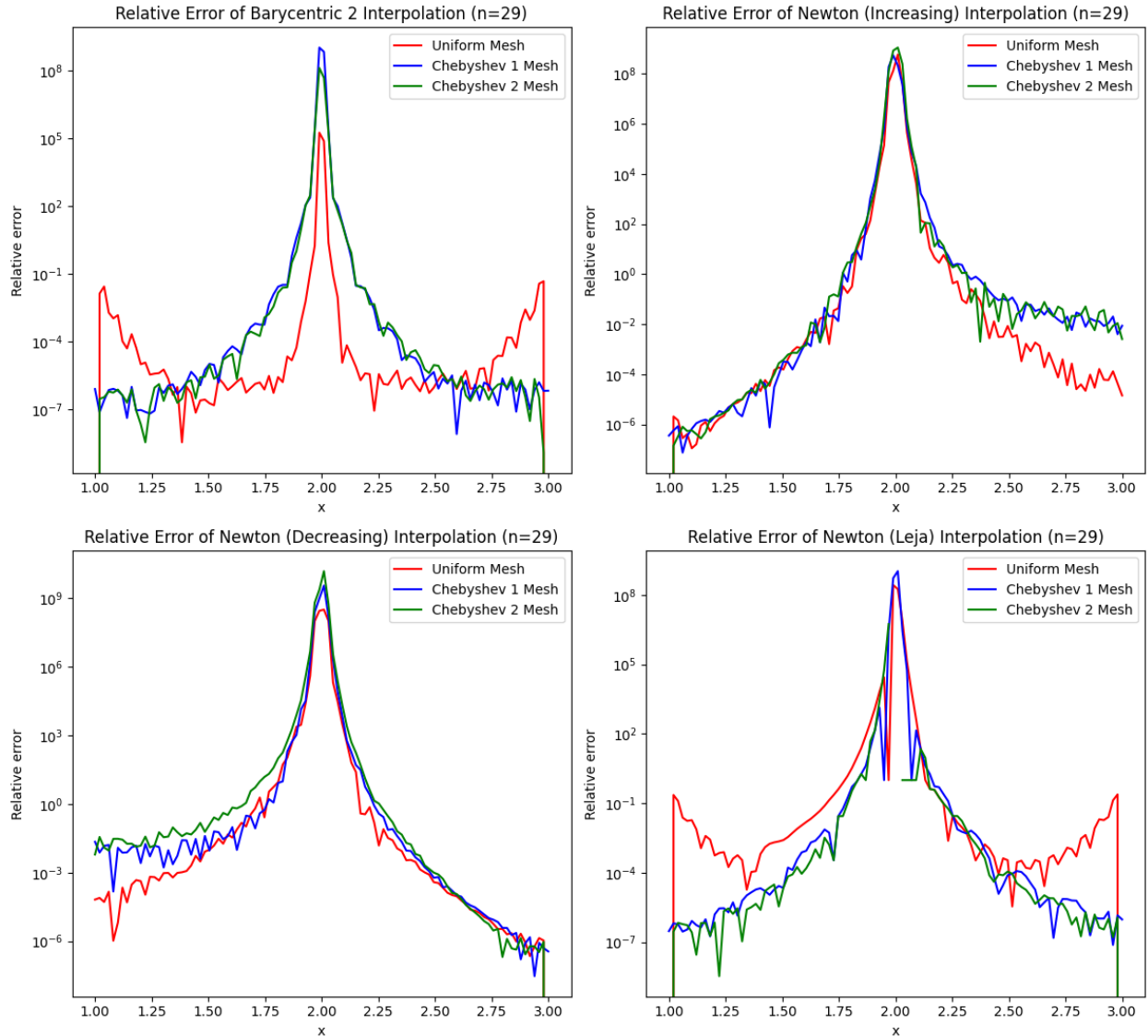


Figure 7. Relative error plots for Barycentric 2 and Newton interpolation with different mesh orderings. All three mesh types are tested for each interpolation strategy. An interpolating polynomial of degree 29 is used. Note the log scale used for the plots.

5.3 Task 3

In this section, interpolation accuracy, conditioning, and stability is discussed for

$$f_2(x; d) = \prod_{i=1}^d (x - i).$$

5.3.1 Interpolation Accuracy

Figure 8 shows attempts to interpolate $f_2(x; 5) = \prod_{i=1}^5 (x - i)$ with $p_{21}(x)$ using Barycentric 1, Barycentric 2, and Newton interpolation methods. Newton is tested with all three mesh types and all tests incorporate all three mesh types. The interpolant is evaluated at 100 equally spaced points on the common interval $[-1, 3]$. When looking at the plots, the interpolation looks quite accurate for all tests and all mesh types. However the scale is quite large, so it is difficult to assess pointwise errors. Table 8 shows the maximum pointwise error for each interpolation strategy with each mesh. Relatively high maximum pointwise errors can be seen when the uniform mesh is used, but when using either of the Chebyshev meshes, maximum pointwise errors are kept to an order of 10^{-4} or 10^{-5} . This relatively high level of accuracy can be explained by the fact that we are using a 21st degree interpolating polynomial to approximate a 5th degree function. Since $n > d$ in this case, the interpolation should be accurate.

$f_2(x)$ Interpolation with Different Strategies and Meshes ($n=21$, $d=5$)

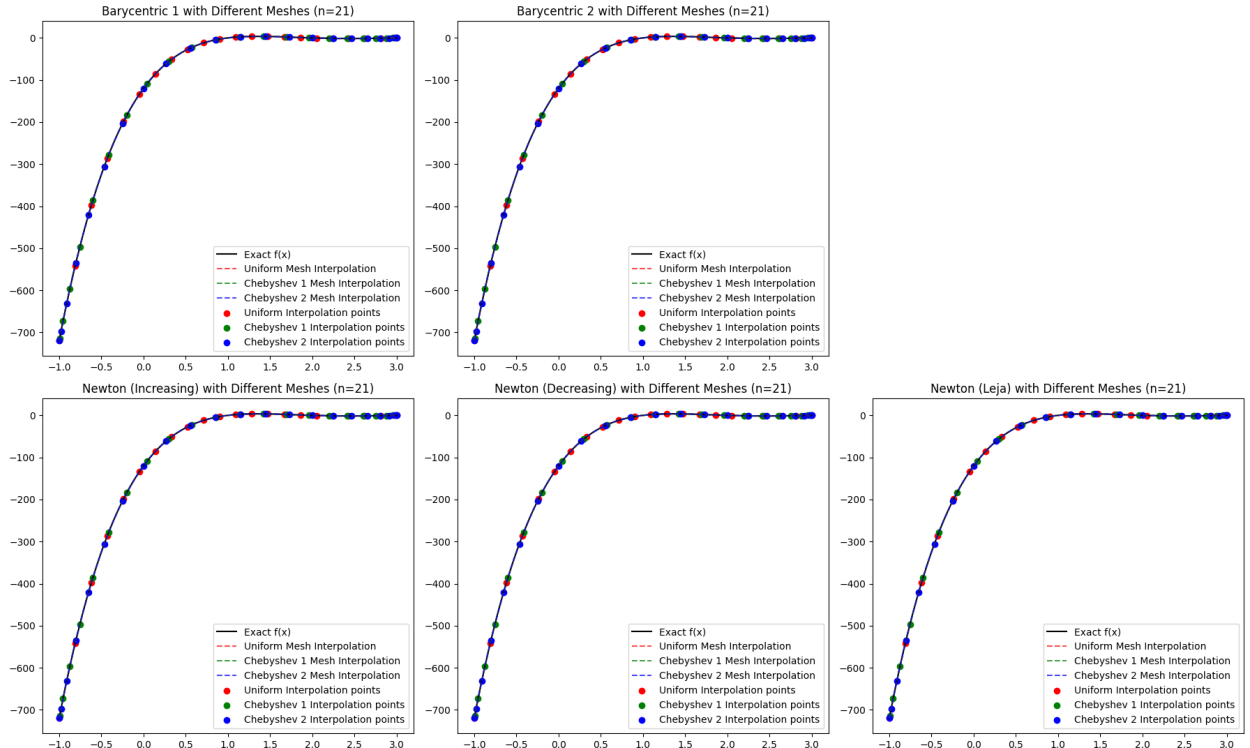


Figure 8. $p_{21}(x)$ interpolating $f_2(x; 5) = \prod_{i=1}^5 (x - i)$. Plots show Barycentric 1, Barycentric 2, Newton interpolation using all 3 mesh types. Newton interpolation is tested for all 3 mesh orders.

Method / Mesh	Maximum Pointwise Error ($\ f(x) - p_n(x)\ _\infty$)
Barycentric 1 (Uniform)	0.0090
Barycentric 1 (Chebyshev 1)	2.5460×10^{-4}
Barycentric 1 (Chebyshev 2)	1.5142×10^{-4}
Barycentric 2 (Uniform)	0.1561
Barycentric 2 (Chebyshev 1)	9.0331×10^{-5}
Barycentric 2 (Chebyshev 2)	7.5898×10^{-5}
Newton Increasing (Uniform)	0.0016
Newton Increasing (Chebyshev 1)	9.7656×10^{-4}
Newton Increasing (Chebyshev 2)	7.3242×10^{-4}
Newton Decreasing (Uniform)	0.0013
Newton Decreasing (Chebyshev 1)	9.9257×10^{-5}
Newton Decreasing (Chebyshev 2)	9.0357×10^{-5}
Newton Leja (Uniform)	0.1271
Newton Leja (Chebyshev 1)	9.7226×10^{-5}
Newton Leja (Chebyshev 2)	9.0331×10^{-5}

Table 8: Maximum Pointwise Error for Each Interpolation Routine ($d = 5, n = 21$)

Figure 9 shows attempts to interpolate $f_2(x; 8) = \prod_{i=1}^8 (x - i)$ with $p_4(x)$ using Barycentric 1, Barycentric 2, and Newton interpolation methods. Newton is tested with all three mesh orderings and all tests incorporate all three mesh types. The interpolant is evaluated at 100 equally spaced points on the common interval $[-1, 3]$. In this series of tests, some clear error can be observed in the plots, even with the large scale. Table 9 shows that the maximum pointwise error is of order 10^3 for all interpolation methods and mesh types. Similarly to a test in Section 5.2.1, no matter what method is used, the maximum error is almost equal for each type of mesh. This is because we are attempting to interpolate an 8th degree function with a 4th degree polynomial. The fundamental limitation in this scenario is the degree of the interpolating polynomial rather than the method used.

$f_2(x)$ Interpolation with Different Strategies and Meshes ($n=4$, $d=8$)

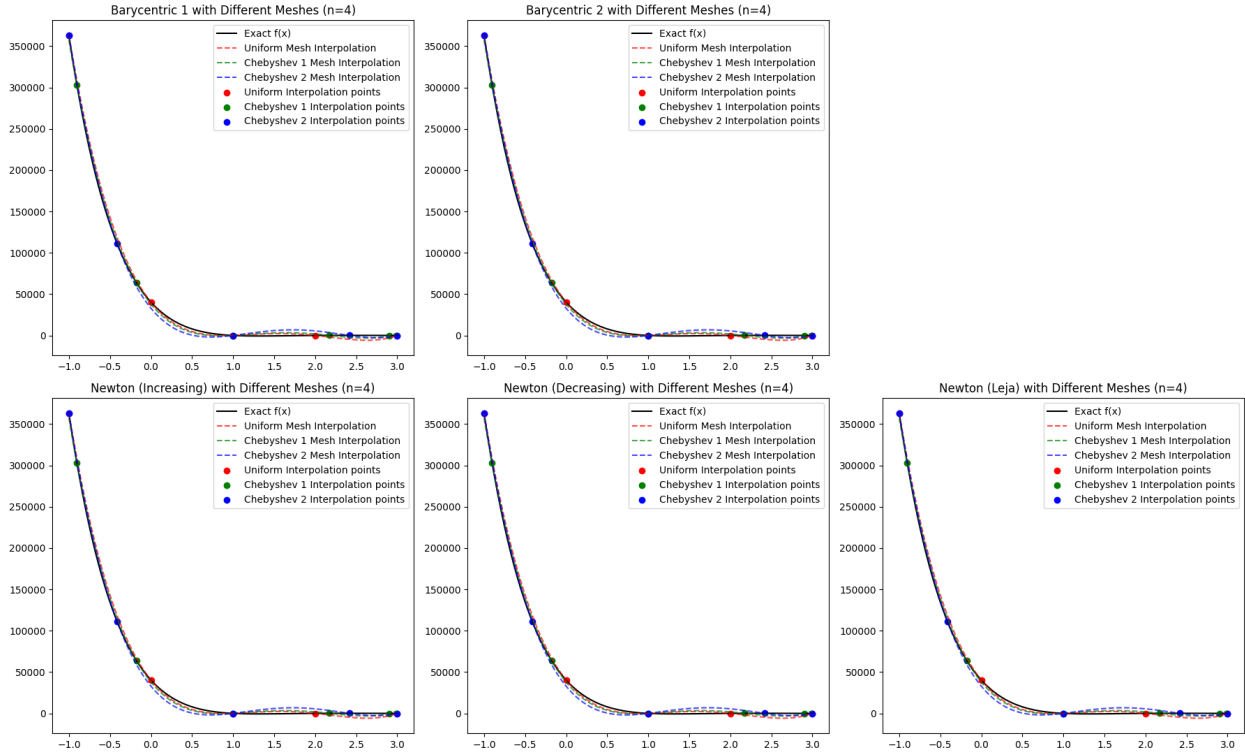


Figure 9. $p_4(x)$ interpolating $f_2(x; 8) = \prod_{i=1}^8 (x - i)$. Plots show Barycentric 1, Barycentric 2, Newton interpolation using all 3 mesh types. Newton interpolation is tested for all 3 mesh orders.

Method / Mesh	Maximum Pointwise Error ($\ f(x) - p_n(x)\ _\infty$)
Barycentric 1 (Uniform)	9861.4931
Barycentric 1 (Chebyshev 1)	5747.9688
Barycentric 1 (Chebyshev 2)	8857.7702
Barycentric 2 (Uniform)	9861.4931
Barycentric 2 (Chebyshev 1)	5748.0000
Barycentric 2 (Chebyshev 2)	8857.7634
Newton Increasing (Uniform)	9861.5087
Newton Increasing (Chebyshev 1)	5748.0000
Newton Increasing (Chebyshev 2)	8857.7302
Newton Decreasing (Uniform)	9861.4775
Newton Decreasing (Chebyshev 1)	5747.9688
Newton Decreasing (Chebyshev 2)	8857.7683
Newton Leja (Uniform)	9861.4931
Newton Leja (Chebyshev 1)	5748.0313
Newton Leja (Chebyshev 2)	8857.7595

Table 9: Maximum Pointwise Error for Each Interpolation Routine ($d = 8$, $n = 4$)

Figure 10 shows attempts to interpolate $f_2(x; 8) = \prod_{i=1}^8 (x - i)$ with $p_{25}(x)$ using Barycentric 1, Barycentric 2, and Newton interpolation methods. Newton is tested with all three mesh

orderings and all tests incorporate all three mesh types. The interpolant is evaluated at 100 equally spaced points on the common interval $[-1, 3]$. In these tests, $n > d$, however, we still see some high maximum pointwise errors in Table 10, specifically when the uniform mesh is used. These high errors with the uniform mesh are likely due to Runge's phenomenon, which occurs when interpolating a function with a high degree polynomial over a set of uniformly spaced nodes. This often leads to oscillatory behavior near the edges of the interval, resulting in large deviations from the true function. The issue is exacerbated here because the interpolant, $p_{25}(x)$, is of much higher degree than the original function. In contrast, interpolation using one of the Chebyshev meshes distributes more nodes near the interval endpoints, which helps control these oscillations and improve accuracy. Some small errors can still be observed with these meshes, but they are far more accurate than the uniform mesh.

$f_2(x)$ Interpolation with Different Strategies and Meshes ($n=25, d=8$)

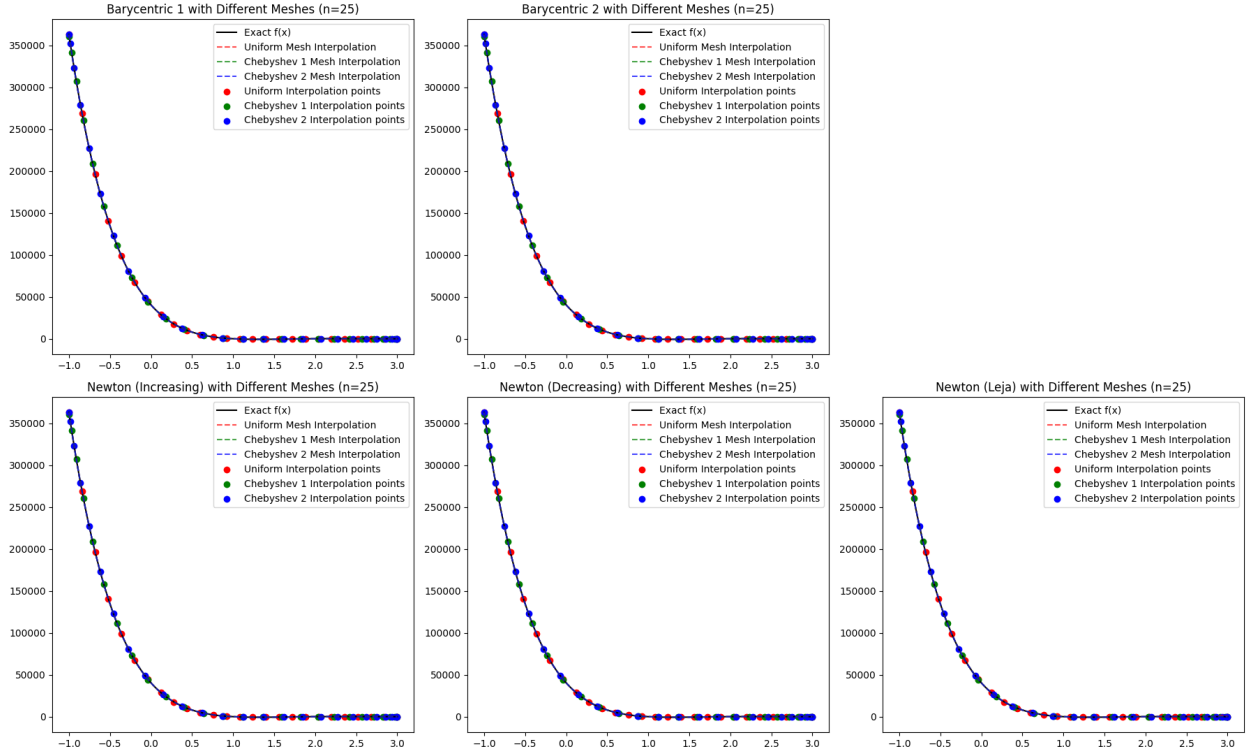


Figure 10. $p_{25}(x)$ interpolating $f_2(x; 8) = \prod_{i=1}^8 (x - i)$. Plots show Barycentric 1, Barycentric 2, Newton interpolation using all 3 mesh types. Newton interpolation is tested for all 3 mesh orders.

Method / Mesh	Maximum Pointwise Error ($\ f(x) - p_n(x)\ _\infty$)
Barycentric 1 (Uniform)	38.5833
Barycentric 1 (Chebyshev 1)	0.1006
Barycentric 1 (Chebyshev 2)	0.1059
Barycentric 2 (Uniform)	1037.8645
Barycentric 2 (Chebyshev 1)	0.0625
Barycentric 2 (Chebyshev 2)	0.0432
Newton Increasing (Uniform)	10.1667
Newton Increasing (Chebyshev 1)	23.5733
Newton Increasing (Chebyshev 2)	115.3571
Newton Decreasing (Uniform)	10.1355
Newton Decreasing (Chebyshev 1)	0.0694
Newton Decreasing (Chebyshev 2)	0.0720
Newton Leja (Uniform)	1865.0832
Newton Leja (Chebyshev 1)	0.0625
Newton Leja (Chebyshev 2)	0.0961

Table 10: Maximum Pointwise Error for Each Interpolation Routine ($d = 8, n = 25$)

5.3.2 Conditioning

Conditioning was tested for Barycentric 1 and Barycentric 2 interpolation for all three mesh types. Conditioning was evaluated using $\kappa(x, n, y)$ and $\kappa(x, n, 1)$, as defined in Section 3. For both barycentric methods and for each mesh type, $\kappa(x, n, y)$ is plotted against the x values to be interpolated, as this behavior can be quite interesting. For Barycentric 1, the Lebesgue constant, or the maximum value of $\kappa(x, n, 1)$, is collected as a summary statistic in Table 11. For previously discussed reasons, the Lebesgue constant is not computed for Barycentric 2. Figure 11 shows $\kappa(x, n, y)$ for both barycentric methods for different n values, using the uniform mesh. Large condition numbers, particularly when using Barycentric 1 can be observed. This highlights the numerical instability of this function and the uniform mesh. Notably, the condition numbers get worse as n increases, specifically at the edges. This is a direct consequence of Runge’s phenomenon. The exponentially increasing Lebesgue constants seen in Table 11 further demonstrate the numerical instability of the uniform mesh.

$\kappa(x, n, y)$ using Barycentric 1 and Barycentric 2 with Uniform Mesh for $f_2(x)$

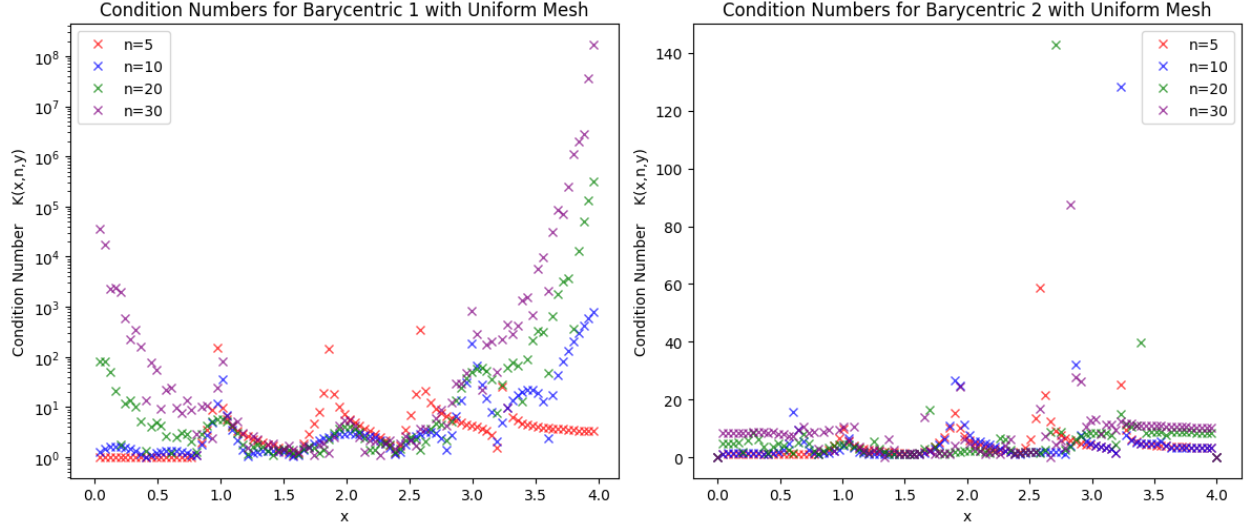


Figure 11. Condition numbers $\kappa(x, n, y)$ for Barycentric 1 and Barycentric 2 interpolation of $f_2(x; 8)$ for different interpolating degrees using the uniform mesh. A log scale is used for the Barycentric 1 plot (left).

Method	$n = 5$	$n = 10$	$n = 20$	$n = 30$
Barycentric 1 Lebesgue Constant	3.1039	29.8970	1.0714×10^4	6.2769×10^6

Table 11: Barycentric 1 Lebesgue Constant for Different Values of n using the uniform mesh

Figure 12 shows $\kappa(x, n, y)$ for both barycentric methods for different n values, using a mesh of Chebyshev points of the first kind. Large condition numbers can still be seen, but they are significantly smaller than those computed when using the uniform mesh, particularly with Barycentric 1. There are noticeable increases in $\kappa(x, n, y)$ near the roots of the $f_2(x; 8)$ ($x = 1, 2, 3, 4$). This is due to the aforementioned division by $|p_n(x)|$ in the computation of the condition number. When close to a root, this is a near-division by 0. The steady Lebesgue constants in Table 12 demonstrate the numerical stability of interpolation with the Chebyshev 1 mesh. The relatively high $\kappa(x, n, y)$ for $n > d$ show the possible downsides of interpolating a function with a polynomial of a much higher degree.

$k(x,n,y)$ using Barycentric 1 and Barycentric 2 with Chebyshev 1 Mesh for $f_2(x)$

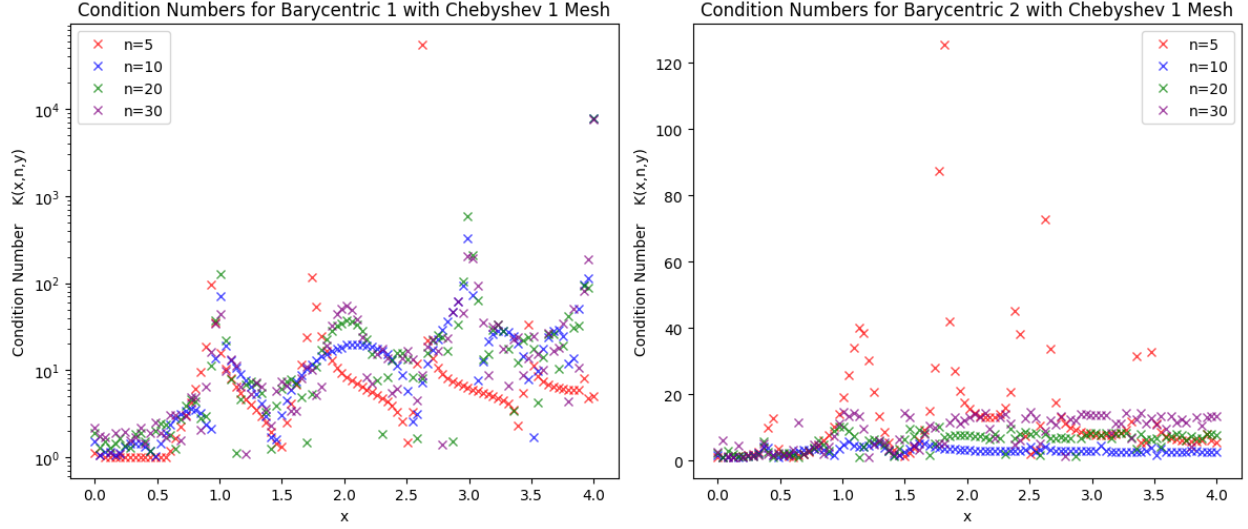


Figure 12. Condition numbers $\kappa(x, n, y)$ for Barycentric 1 and Barycentric 2 interpolation of $f_2(x; 8)$ for different interpolating degrees using the Chebyshev 1 mesh. A log scale is used for the Barycentric 1 plot (left).

Method	$n = 5$	$n = 10$	$n = 20$	$n = 30$
Barycentric 1 Lebesgue Constant	2.1044	2.4894	2.9008	3.1487

Table 12: Barycentric 1 Lebesgue Constant for Different Values of n using the Chebyshev 1 mesh

Figure 13 shows $\kappa(x, n, y)$ for both barycentric methods for different n values, using a mesh of Chebyshev points of the first kind. Somewhat large condition numbers can still be seen, but again they are smaller than those computed when using the uniform mesh. We see similar behavior near the roots of $f_2(x; 8)$, with $\kappa(x, n, y)$ showing large increases due to the inherent division by a near-zero value. The steady Lebesgue constants in Table 13 demonstrate the numerical stability of interpolation with a mesh of Chebyshev points of the second kind. The majority of the numerical instability here is from the instability of the function, rather than the interpolation method or the Chebyshev mesh.

$k(x,n,y)$ using Barycentric 1 and Barycentric 2 with Chebyshev 2 Mesh for $f_2(x)$

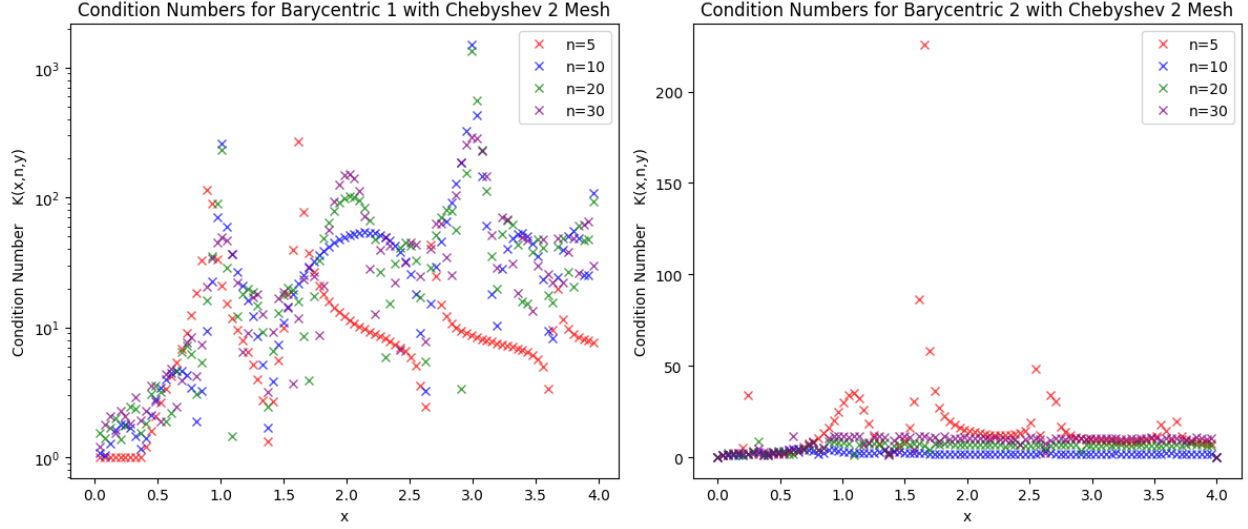


Figure 13. Condition numbers $\kappa(x, n, y)$ for Barycentric 1 and Barycentric 2 interpolation of $f_2(x; 8)$ for different interpolating degrees using the Chebyshev 2 mesh. A log scale is used for the Barycentric 1 plot (left).

Method	$n = 5$	$n = 10$	$n = 20$	$n = 30$
Barycentric 1 Lebesgue Constant	1.9877	2.4196	2.8516	3.1239

Table 13: Barycentric 1 Lebesgue Constant for Different Values of n using the Chebyshev 2 mesh

5.3.3 Relative Error and Stability

Relative error is computed using the method in Higham (2004), as previously discussed. Figure 14 shows relative error results for $f_2(x; 8) = \prod_{i=1}^8 (x - i)$ on the interval $[0, 10]$. The relative error when using Barycentric 2 is generally pretty low, around 10^{-5} or 10^{-6} . However, the relative error increases rapidly at the edges of the interval when using a uniform mesh. This is a result of the erratic oscillations at the endpoints when attempting to interpolate with a uniform mesh. The relative error gets rather large when computing Newton interpolation with an increasing or decreasing mesh order, regardless of mesh type. There seem to be increases in the relative error at the roots of the function (the integers less than or equal to 8). This is due to the division by $|p_n(x)|$ in the computation of relative error. When close to the root, this is effectively dividing by a number very close to 0, which causes the relative error to increase. This highlights the numerical instability of these routines around the roots of a polynomial. The Leja ordering does slightly better than the increasing or decreasing order, but erratic oscillations can still be observed, especially when using the uniform mesh.

Relative Error with Different Strategies and Meshes (n=25)

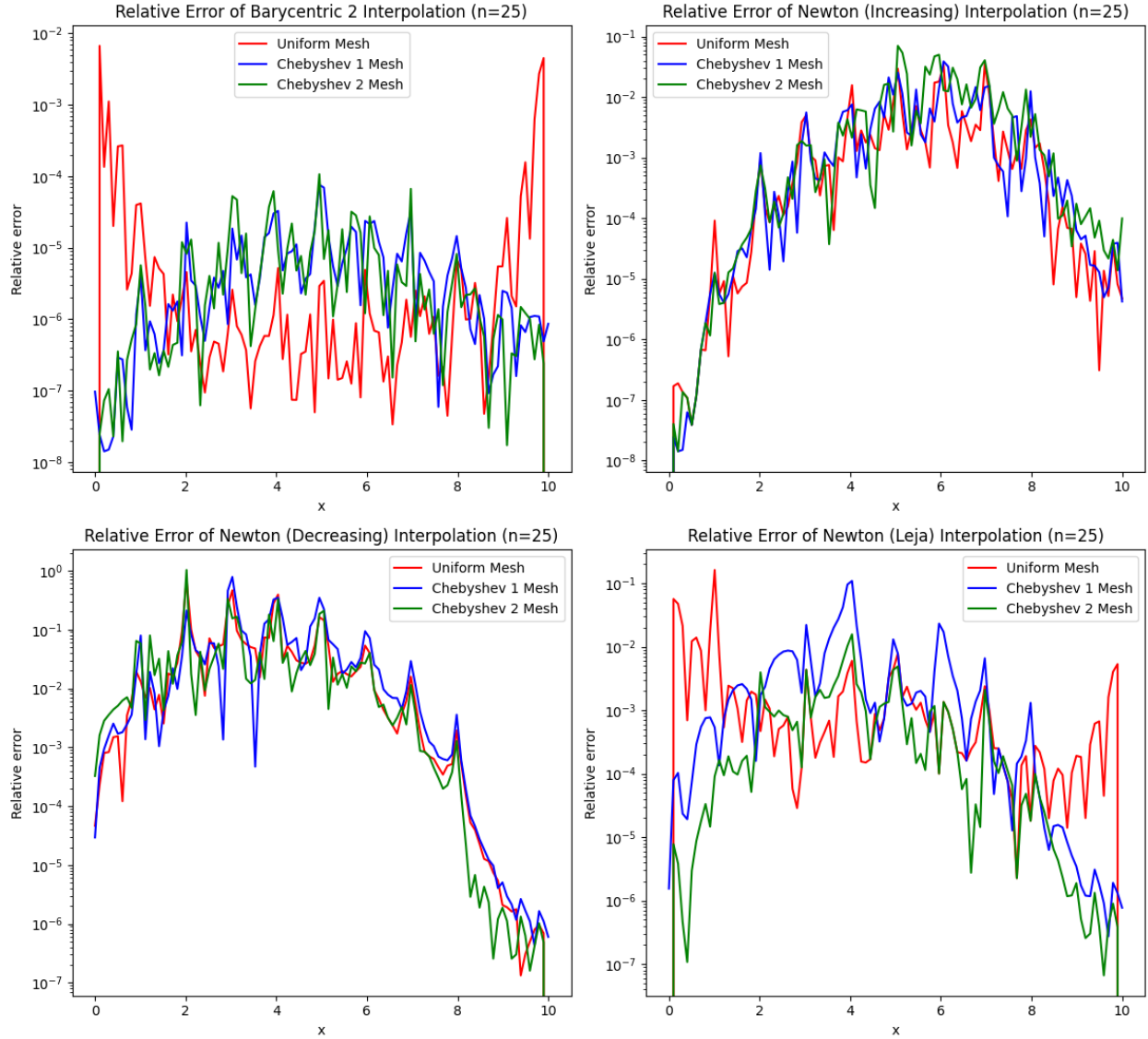


Figure 14. Relative errors in computed $p_n(X)$ for Barycentric 2 and Newton interpolation with different mesh orderings on $[0, 10]$. All three mesh types are tested for each interpolation strategy. An interpolating polynomial of degree 25 is used and $f_2(x; 8) = \prod_{i=1}^8 (x - i)$. Note the log scale used for the plots.

5.4 Task 4

In this section we discuss the interpolation of $f_3(x) = \ell_n(x)$, where $f_3(x_i) = 0$, $0 \leq i \leq n$ and $f_3(x_n) = 1$. So $\ell_n(x)$ is a Lagrange basis function. This is an interesting function since $f(x)$ itself is determined by the mesh. Thus, no matter what mesh is used, the interpolating polynomial will always accurately approximate $f(x)$. Figure 15 shows attempts to interpolate $f_3(x)$ with $p_5(x)$ using Barycentric 1 and Barycentric 2 with all three mesh types. The interpolant is evaluated at 100 equally spaced points on the common interval $[-2, 3]$. It

can be observed that there is no error in the plots, no matter what mesh is used. Further experimentation, which unfortunately did not come to fruition for this assignment, could be undertaken to examine if this relationship holds for Newton interpolation with different mesh orderings. Further work could also be done to generate data regarding condition numbers and relative error for $f_3(x)$. According to Higham (2004), Barycentric 1 should perform stably, while Barycentric 2 should perform unstably. Higham also shows that Newton interpolation performs more stably when using an increasing mesh order than when using a decreasing mesh order.

$f_3(x)$ Interpolation with Different Strategies and Meshes ($n=5$)

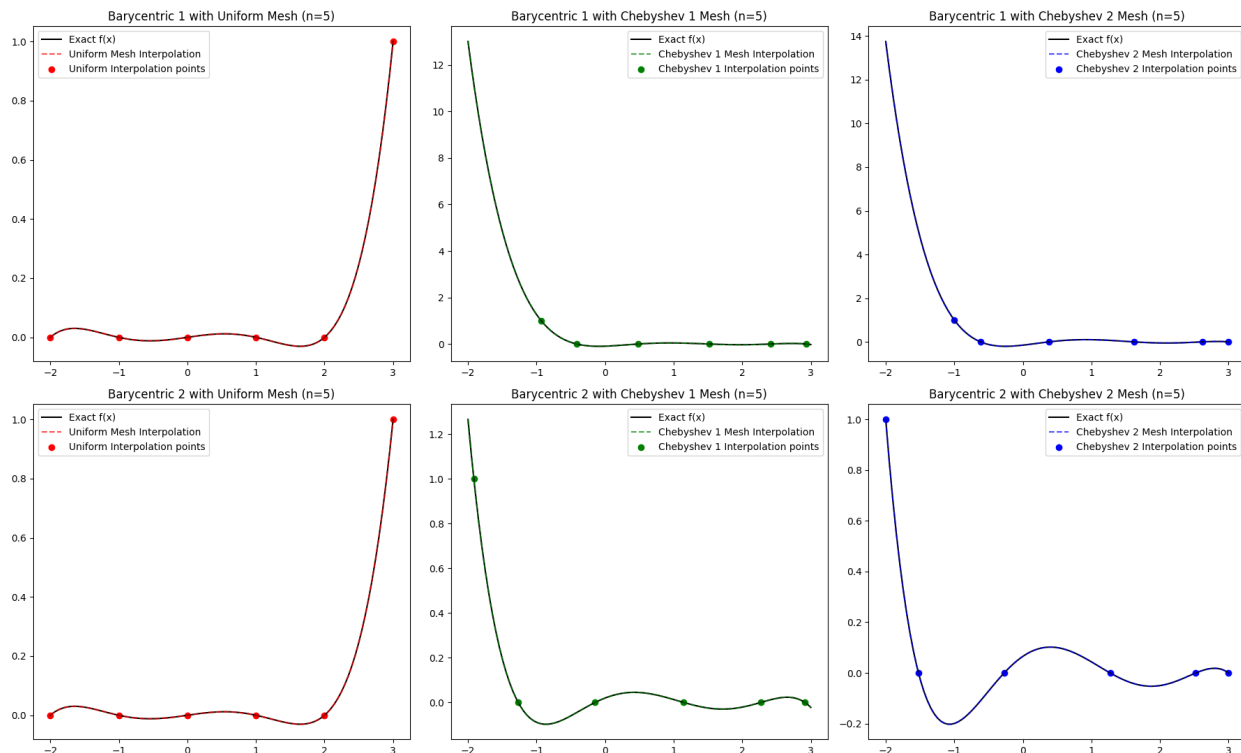


Figure 15. $p_5(x)$ interpolating $f_3(x) = \ell_n(x)$. Plots show Barycentric 1 and Barycentric 2 interpolation using all 3 mesh types.

5.5 Task 5

5.5.1 Convergence

We will first investigate the convergence (or lack thereof) to $f_4(x) = 1/(1 + 25x^2)$ as n increases for the Barycentric Form 2 with the uniform points and Chebyshev points of the first and second kind. Figure 16 shows the results of various tests to assess convergence to the Runge function. Table 14 shows the maximum pointwise error at each degree for each of the mesh types. When using the uniform mesh, we see a classic example of the (appropriately named) Runge Phenomenon. As n increases, it appears that the interpolating polynomial is converging to the function when looking at the center of the interval. However, there are massive oscillations towards the edges, which increase in scale exponentially as n increase.

This clearly demonstrates the disastrous effects of using the uniform mesh to interpolate functions such as this one. On the other hand, interpolation using either of the Chebyshev meshes can be seen converging to the Runge function. Observation of Table 14 shows the rapid decrease of the maximum pointwise error when using the Chebyshev meshes. With 50 mesh points, we already see maximum pointwise errors of the order of 10^{-5} . Increase to $n = 100$ and one observes maximum pointwise errors of the order of 10^{-7} . Interestingly, this seems to be a limit that we cannot go much past. When using 250 mesh points, the maximum pointwise error for both kinds of Chebyshev meshes increases from the $n = 100$ case.

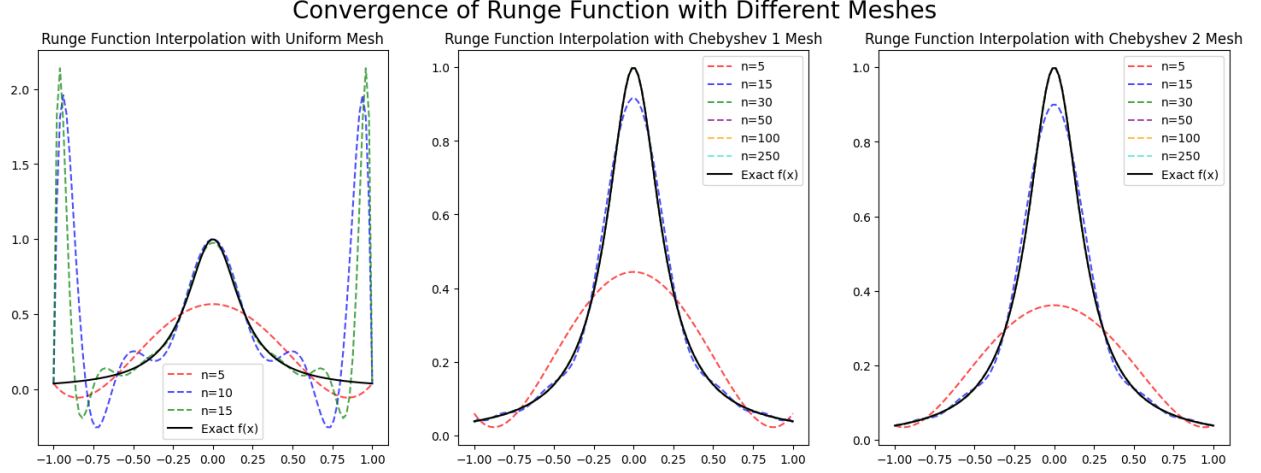


Figure 16. Interpolation of the Runge function for all three mesh types using interpolating polynomials of varying degrees to assess convergence (or lack thereof).

Mesh Type	$n = 5$	$n = 10$	$n = 15$	$n = 30$	$n = 50$	$n = 100$	$n = 250$
Uniform	0.4303	1.9154	2.0967	-	-	-	-
Chebyshev 1	0.5535	-	0.08182	0.002054	3.9605×10^{-5}	4.8731×10^{-7}	1.1066×10^{-6}
Chebyshev 2	0.6362	-	0.09793	0.002379	4.4194×10^{-5}	2.8446×10^{-7}	6.8303×10^{-7}

Table 14: Maximum Pointwise Error for Each Mesh Type and Polynomial Degree for Runge Function

5.5.2 Conditioning

Conditioning was tested for Barycentric 1 and Barycentric 2 interpolation for all three mesh types. Conditioning was evaluated using $\kappa(x, n, y)$ and $\kappa(x, n, 1)$, as defined in Section 3. For both barycentric methods and for each mesh type, $\kappa(x, n, y)$ is plotted against the x values to be interpolated, as this behavior can be quite interesting. The mean $\kappa(x, n, y)$ over the given interval is noted as a summary statistic. For Barycentric 1, the Lebesgue constant, or the maximum value of $\kappa(x, n, 1)$, is computed as well. Figure 17 shows $\kappa(x, n, y)$ for both barycentric methods for different n values, using the uniform mesh. Using Barycentric 1, we see somewhat high condition numbers, especially as you get further away from $x = 2$. This lines up with the uniform mesh's poor performance at the ends of the interval. The $\kappa(x, n, y)$ values are lower when using Barycentric 2, however, there are still some high outliers. The

exponentially increasing Lebesgue constants seen in Table 15 demonstrate the ill-conditioned nature of interpolating with a uniform mesh.

$\kappa(x,n,y)$ using Barycentric 1 and Barycentric 2 with Uniform Mesh for $f_4(x)$

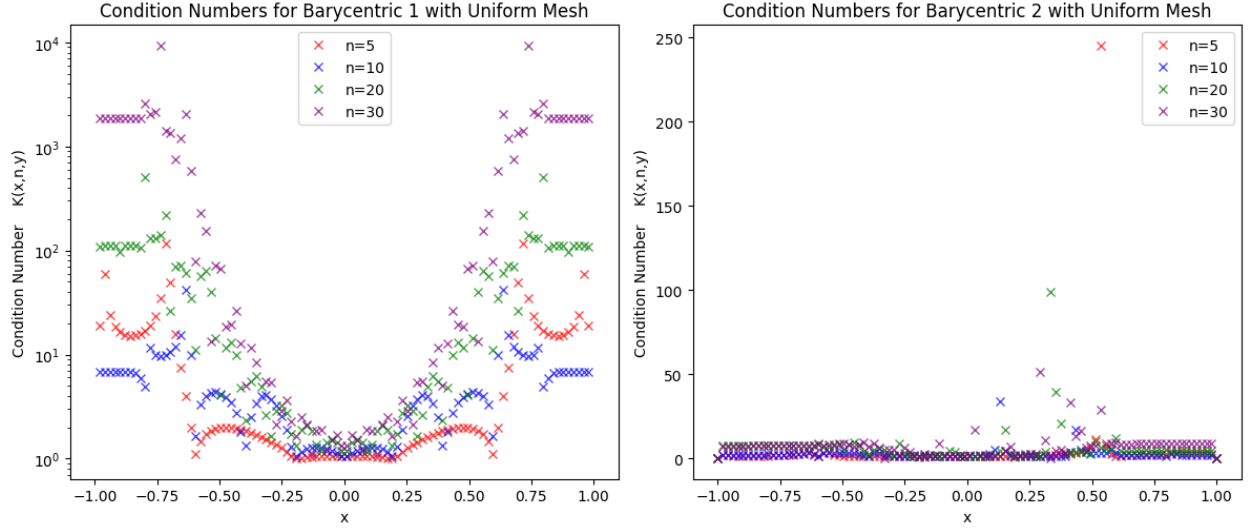


Figure 17. Condition numbers $\kappa(x, n, y)$ for Barycentric 1 and Barycentric 2 interpolation of $f_4(x)$ for different interpolating degrees using the uniform mesh. A log scale is used for the Barycentric 1 plot (left).

Method	$n = 5$	$n = 10$	$n = 20$	$n = 30$
Barycentric 1 Mean $K(x, n, y)$	10.6288	5.3994	53.0697	820.5304
Barycentric 1 Lebesgue Constant	3.1039	29.8970	1.0714×10^4	6.2769×10^6
Barycentric 2 Mean $K(x, n, y)$	4.4399	2.6099	6.3266	7.2289

Table 15: Comparison of Mean $K(x, n, y)$ and Lebesgue Constant for Barycentric 1 and Barycentric 2 using the uniform mesh

Figure 18 shows $\kappa(x, n, y)$ for both barycentric methods for different n values, using a mesh of Chebyshev points of the first kind. Condition numbers $\kappa(x, n, y)$ with this mesh choice are lower for Barycentric 1 and around the same for Barycentric 2. The mean $\kappa(x, n, y)$ value for each n , as seen in Table 16, stays below 10, demonstrating that this function is somewhat well-conditioned when using the Chebyshev 1 mesh. Another key observation from Table 16 is the steady Lebesgue constants as n increases. This highlights the clear well-conditioned nature of interpolating with this choice of mesh.

$k(x,n,y)$ using Barycentric 1 and Barycentric 2 with Chebyshev 1 Mesh for $f_4(x)$

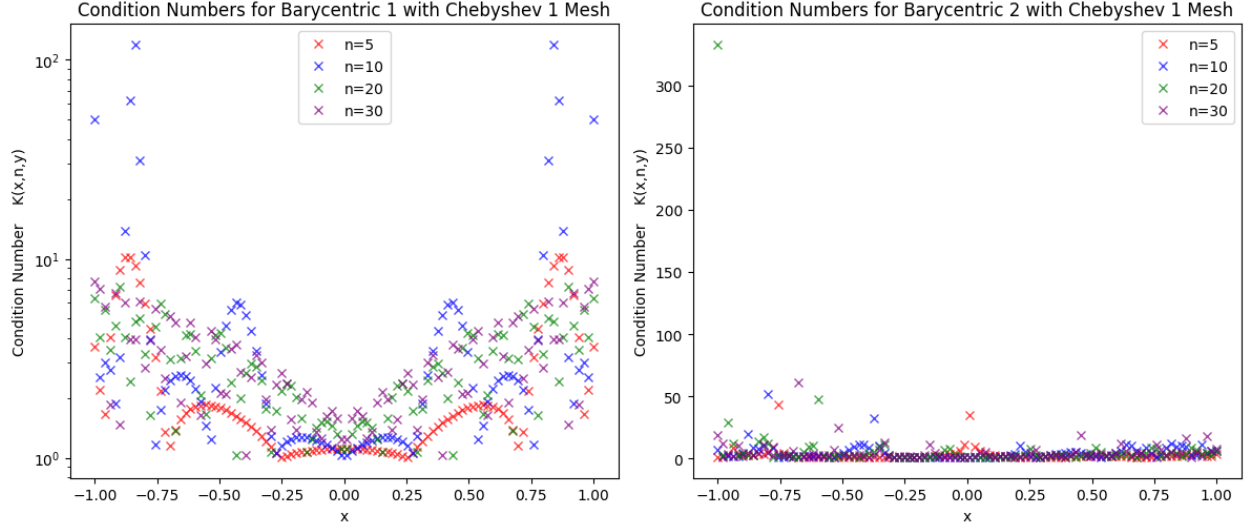


Figure 18. Condition numbers $\kappa(x, n, y)$ for Barycentric 1 and Barycentric 2 interpolation of $f_4(x)$ for different interpolating degrees using the Chebyshev 1 mesh. A log scale is used for the Barycentric 1 plot (left).

Method	$n = 5$	$n = 10$	$n = 20$	$n = 30$
Barycentric 1 Mean $K(x, n, y)$	2.5854	7.8614	2.9723	3.1677
Barycentric 1 Lebesgue Constant	2.1044	2.4894	2.9008	3.1487
Barycentric 2 Mean $K(x, n, y)$	3.0328	4.9754	7.6758	5.0249

Table 16: Comparison of Mean $K(x, n, y)$ and Lebesgue Constant for Barycentric 1 and Barycentric 2 using the Chebyshev 1 mesh

Figure 19 shows $\kappa(x, n, y)$ for both barycentric methods for different n values, using a mesh of Chebyshev points of the second kind. Low condition numbers $\kappa(x, n, y)$ can be seen for both barycentric methods. Additionally, as n increases, the Lebesgue constant remains steady, as seen in Table 17. This demonstrates the well-conditioned nature of this interpolating strategy. Overall, we observed that condition numbers of both forms, $\kappa(x, n, y)$ and $\kappa(x, n, 1)$, were quite a bit lower when using either of the Chebyshev meshes, compared to the uniform mesh. This backs up what was observed in the convergence results. Interpolation the Runge function should be done with a mesh of Chebyshev points.

$\kappa(x,n,y)$ using Barycentric 1 and Barycentric 2 with Chebyshev 2 Mesh for $f_4(x)$

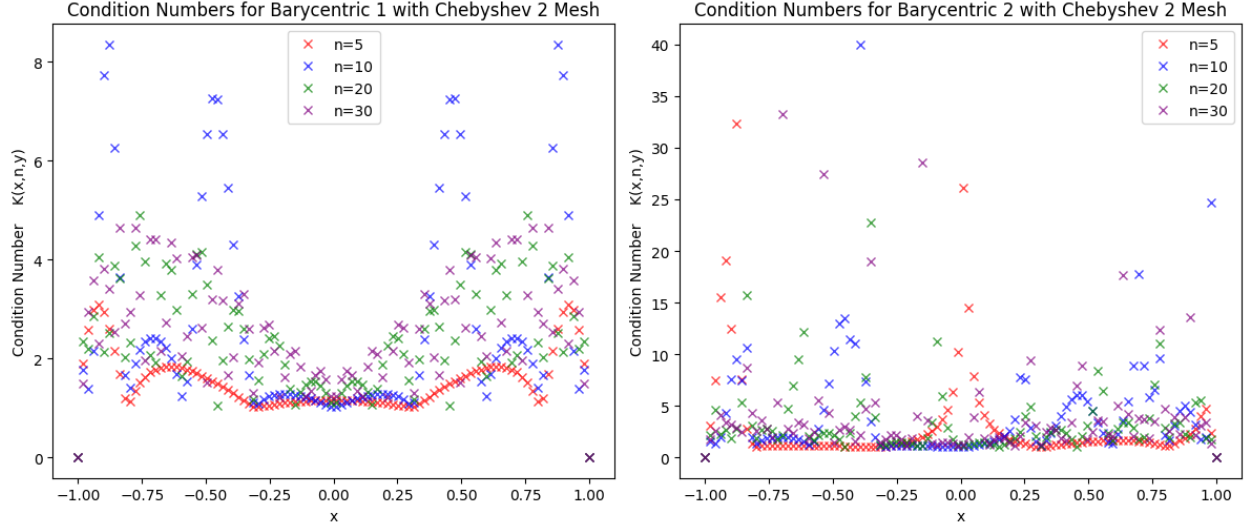


Figure 19. Condition numbers $\kappa(x, n, y)$ for Barycentric 1 and Barycentric 2 interpolation of $f_4(x)$ for different interpolating degrees using the Chebyshev 2 mesh.

Method	$n = 5$	$n = 10$	$n = 20$	$n = 30$
Barycentric 1 Mean $K(x, n, y)$	1.5212	2.7374	2.4447	2.6072
Barycentric 1 Lebesgue Constant	1.9877	2.4196	2.8516	3.1239
Barycentric 2 Mean $K(x, n, y)$	3.0733	4.2360	3.2981	4.1949

Table 17: Comparison of Mean $K(x, n, y)$ and Lebesgue Constant for Barycentric 1 and Barycentric 2 using the Chebyshev 2 mesh

5.5.3 Relative Error and Stability

Relative error is computed as the relative change between the interpolating polynomial computed in single precision and the "exact" interpolating polynomial computed in double precision, as described earlier. Figure 20 shows relative error results for $f_4(x) = 1/(1 + 25x^2)$. The relative error when using Barycentric 2 is seen to be quite low ($\approx 10^{-7}$), except when using the uniform mesh. The relative error with the uniform mesh reaches as high as 10^{-2} at the edges of the interval. This aligns with the erratic oscillatory behavior seen at the interval edges when interpolating the Runge function with a uniform mesh. The relative error observed when using Newton and either an increasing or decreasing order aligns accurately with the results seen in Higham (2004) (Figures 3 & 4 in the Higham paper). When using an increasing mesh order, the relative error increases on the right side of the interval, almost up to 10^3 . With the decreasing mesh order, the relative error increases to the same large value when going left from $x = 0$. This suggests that numerical conditioning plays a crucial role in determining stability. The Leja ordering provides an improvement over uniform mesh interpolation but remains somewhat erratic when applied to uniform points, indicating sensitivity to node placement. These results emphasize that the choice of mesh remains the most important factor in ensuring stable and reliable interpolation of the Runge function.

Relative Error with Different Strategies and Meshes (n=29)

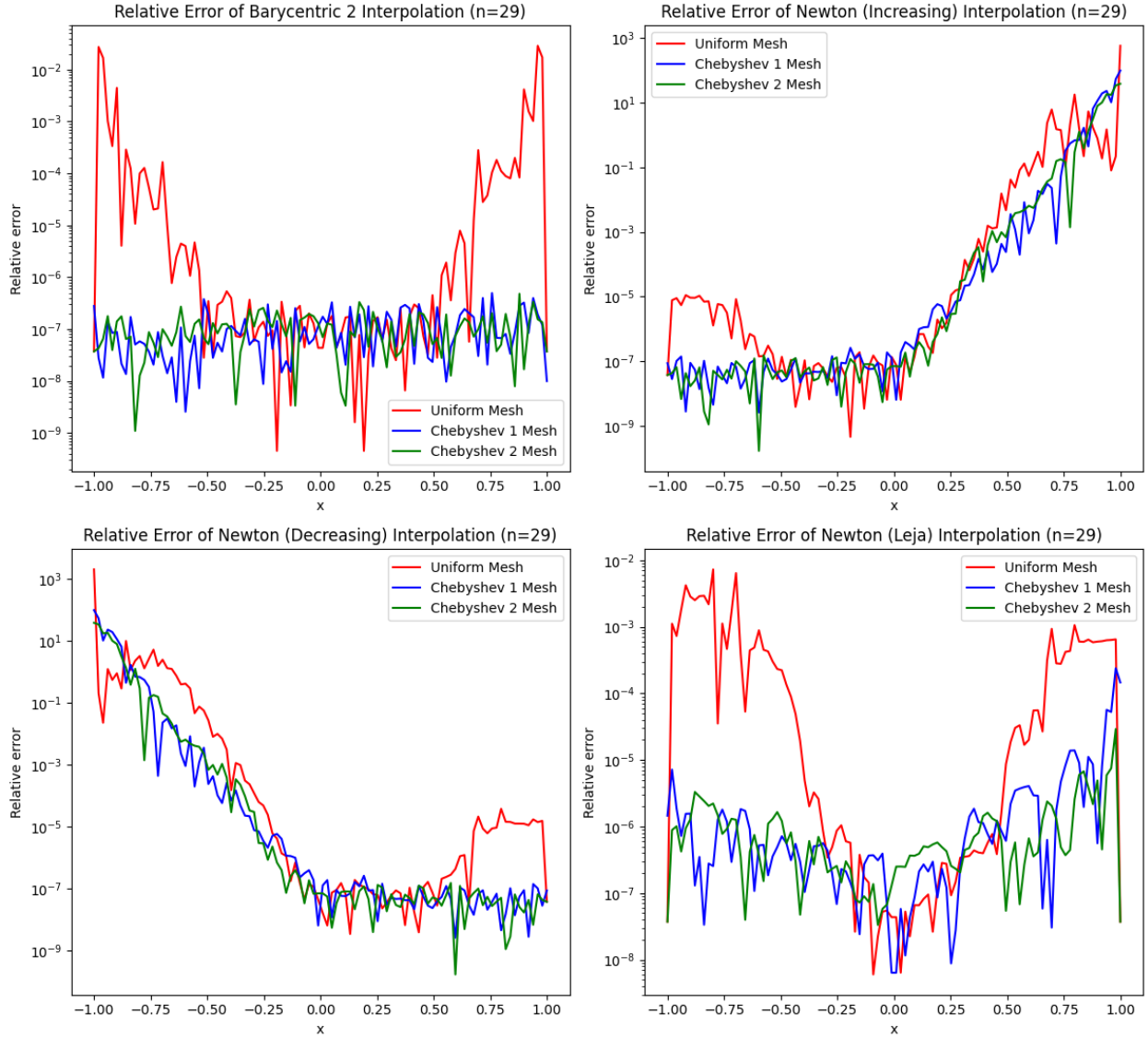


Figure 20. Relative errors in computed $p_n(X)$ for Barycentric 2 and Newton interpolation with different mesh orderings. All three mesh types are tested for each interpolation strategy. An interpolating polynomial of degree 29 is used. Note the log scale used for the plots.

6 Conclusions

This report analyzed the accuracy, conditioning, and stability of Barycentric 1, Barycentric 2, and Newton interpolation methods across different node distributions. A key finding was that uniform node spacing led to significant numerical instability, particularly at high polynomial degrees due to the Runge phenomenon, while Chebyshev nodes consistently improved accuracy and stability. Barycentric Form 2 outperformed Barycentric Form 1 in numerical conditioning, and Newton interpolation exhibited instability unless paired with well-ordered nodes such as the Leja ordering.

Conditioning and relative error analysis confirmed that high-degree interpolation amplifies sensitivity to small numerical errors, particularly near function roots. While all implemented algorithms were theoretically sound, some discrepancies in error trends suggest minor round-off errors or precision limitations. Overall, the study reinforces the importance of choosing the right node distribution and interpolation method to ensure numerical stability, with Chebyshev nodes and Barycentric Form 2 proving to be the most reliable choices.

7 References

In the creation of this report, I referenced Dr. Gallivan's class notes and the papers of Higham and Berrut & Trefethen. I also consulted with Reyajul Hasan Leon, Dan Mazus and Brayton Link about algorithm development.